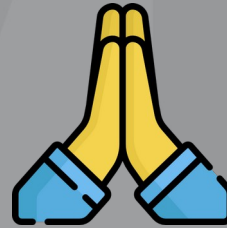


Tutoría Colectiva Final

03MIAR – Algoritmos de optimización



Agenda

1. Evaluación y Entregas de la asignatura
2. Introducción a los algoritmos
3. Algoritmos de Ordenación
4. Técnicas de diseño de algoritmos
5. Problemas Tipo
6. Algoritmos de Búsqueda
7. Descenso del Gradiente
8. Algoritmos Heurísticos
9. Algoritmos Genéticos
10. Bibliografía

Foro. Práctica de Algoritmo genético



Álvaro Gil Natividad NUEVO

Hace 22 horas, a las 13:45

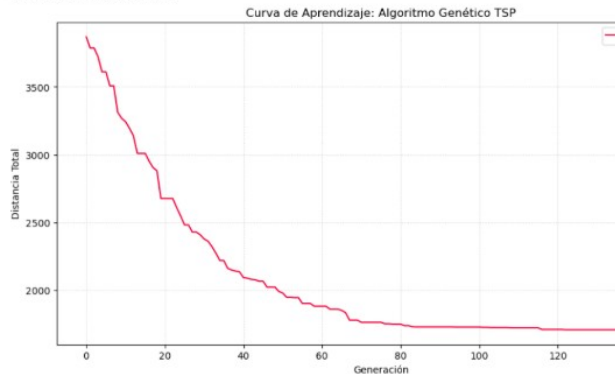
A partir del Código que has compartido de la clase práctica de algoritmos Genéticos , adjunto la gráfica de evolución de mi algoritmo genético. Quería compartirla porque ilustra muy bien cómo funciona el proceso evolutivo:

```
Generación 0: Distancia = 3786
Generación 20: Distancia = 2676
Generación 40: Distancia = 2088
Generación 60: Distancia = 1981
Generación 80: Distancia = 1737
Generación 100: Distancia = 1725
Generación 120: Distancia = 1709
Generación 140: Distancia = 1707
```

RESULTADO FINAL

Mejor Distancia: 1707

Mejor Ruta encontrada: [1, 0, 6, 5, 26, 12, 11, 25, 20, 30, 22, 29, 2, 27, 3, 32, 31, 20, 33, 34, 30, 39, 21, 24, 40, 23, 41, 9, 8, 10, 18, 4, 7, 17, 35, 36, 37, 15, 16, 14, 19, 13]



Como veis en la captura, partimos de una solución aleatoria bastante "mala" (distancia 3786). Sin embargo, gracias a aplicar correctamente la **factibilización** (para no tener ciudades repetidas) y el operador de **mutación**, el fitness mejora drásticamente.

Lo interesante es ver cómo la gráfica deja de caer en picado sobre la generación 60. Eso nos indica que, si queremos mejorar más allá de 1707, quizás necesitaríamos aumentar la población o probar técnicas más avanzadas como una inicialización heurística, ya que el algoritmo parece haber encontrado su límite natural con la configuración actual.

Un saludo.

Foro. Práctica de Algoritmo genético



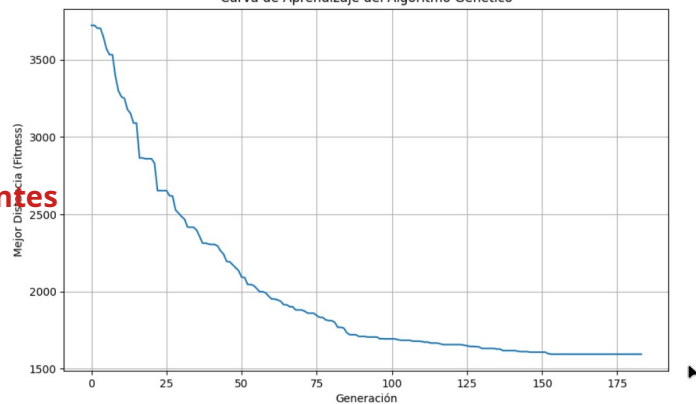
David Pino Molero **NUEVO**
Hace 16 horas, a las 19:44

Buenas tardes.

A partir de las aportaciones de los dos compañeros, especialmente la de Álvaro, creo que es interesante añadir un segundo criterio de parada basado en el número de generaciones sin mejora, de forma que el algoritmo no siga iterando una vez que no se consiga mejora.

Parada por estancamiento: 30 generaciones sin mejora.

Curva de Aprendizaje del Algoritmo Genético



Ojo! Conviene analizar la población. Suele ser habitual que con las iteraciones se “repitan” los individuos genéticamente.

Una opción saludable es eliminar los repetidos en cada iteración antes de seleccionar a los supervivientes.

Evaluación y Entregas

03MIAR – Algoritmos de Optimización

Agenda

1. Evaluación y Entregas de la asignatura
2. Introducción a los algoritmos
3. Algoritmos de Ordenación
4. Técnicas de diseño de algoritmos
5. Problemas Tipo
6. Algoritmos de Búsqueda
7. Descenso del Gradiente
8. Algoritmos Heurísticos
9. Algoritmos Genéticos
10. Bibliografía

Sistema de Evaluación



Sistema de Evaluación	Ponderación
Portafolio	60 %
Trabajo Práctico(*) : 30% Actividades Guiadas(*) : 10% Estudio y análisis de un artículo científico(*): 10% Participación en Foro(Evaluable): 10%	
Sistema de Evaluación	Ponderación
Prueba final*	40 %
<i>Preguntas tipo test con una sola respuesta válida. Las respuestas no correctas restan, las respuestas no contestadas ni suman ni restan.</i>	

***Es requisito indispensable para superar la asignatura aprobar cada apartado (portafolio (*y prueba final) con un mínimo de 5 para ponderar las calificaciones.**

Trabajo práctico.(*). Fechas de entrega

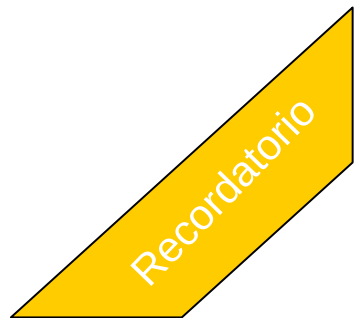


- Desarrollar, modelar y analizar algoritmos según diferentes técnicas para resolver el problema planteado en la asignatura.
- Resolver un problema real. Entrega de PDF
- Deben identificarse los aspectos teóricos en la entrega
- Fecha limite de entrega 1ª convocatoria: 06/03/2026
- Fecha limite de entrega 2ª convocatoria: 17/04/2026

Actividad. Estudio y análisis de artículo científico (*)

Fechas de Entrega

- Lectura, análisis e interpretación de un artículo científico
- Contestar a preguntas sobre el artículo
- Tema: Heurísticas
- Fecha limite de entrega 1ª convocatoria: 06/03/2026 23:59
- Fecha limite de entrega 2ª convocatoria: 17/04/2026 23:59



Foro



- Una cuestión de debate en las 2 primeras semanas: 2 cuestiones.
- Aportaciones que ayuden a los compañeros serán valoradas.
- Cuestiones particulares deben exponerse por correo electrónico.
- No “forzar” la participación ni “eludirla”.
- Leer todas las participaciones y evitar repetir comentarios de otros compañeros.
- Fecha limite de entrega 1ª convocatoria: **06/03/2026**

Examen(*). Fechas de entrega

- Fecha 1ª convocatoria : 06 de marzo a 12:00 y 20:00
- Fecha 2ª convocatoria : 17 de abril a 12:00 y 20:00
- Duración: 1 hora
- 20 preguntas tipo test: Acierto: **+5**. Fallo: **-0.167**



Avisar por correo de cualquier incidencia INMEDIATAMENTE
raul.reyero@professor.universidadviu.com

+

soporte



- ✓ Obligatorio
- ✓ Realizar pasos indicados, cualquier problema contactar con soporte
- ✓ Hacedlo con tiempo

Examen(*).

40%

- Preparar el examen



Contenido de la asignatura

- Visible para los estudiantes ▼
- Recursos y materiales de la asignatura



Recursos de aprendizaje

- Visible para los estudiantes ▼



Manual de la asignatura

- Visible para los estudiantes ▼



03MIAR | ALGORITMOS DE OPTIMIZACIÓN(SCORM)

- Visible para los estudiantes ▼
- Sin fecha de entrega
- Explicación y práctica de las técnicas y métodos para diseñar y analizar algoritmos orientados a resolver problemas de optimización en el ámbito de la inteligencia artificial.

Convocatorias y entregas tardías

- * Si es la **primera convocatoria** de la actividad:

En este caso, la entrega se ignora y se suspende directamente con un 0. El alumno puede presentar el mismo trabajo, si así lo desea, en segunda convocatoria, en cuyo caso se corregirá de forma normal sin penalización.

- * Si es la **segunda convocatoria** de la actividad (pero se ha entregado antes de publicar las actas):

se procederá a corregir la actividad, pero se quedará en 5 sobre 10.

- * Si es la **segunda convocatoria** de la actividad pero se ha entregado tras publicar las actas: se ignorará la entrega y se considerará la actividad como no presentada.

Introducción a los algoritmos

03MIAR – Algoritmos de Optimización

Agenda

1. Evaluación y Entregas de la asignatura
- 2. Introducción a los algoritmos**
3. Algoritmos de Ordenación
4. Técnicas de diseño de algoritmos
5. Problemas Tipo
6. Algoritmos de Búsqueda
7. Descenso del Gradiente
8. Algoritmos Heurísticos
9. Algoritmos Genéticos
10. Bibliografía

¿Qué es un algoritmo?

- **Definición:** Un algoritmo es un conjunto de reglas ordenadas y finitas para realizar una actividad mediante la sucesión de pasos claramente establecidos(*).
- Cada día usamos algoritmos sin darnos cuenta:
 - Operaciones numéricas sencillas(sumas, restas,...)
 - Poner la mesa o cocinar una receta de cocina
 - Elegir una película y buscar el asiento en el cine.
 - Conducir,...
- Centraremos nuestro estudio en los problemas de cálculo y más específicamente a los de optimización.

Tipos de Algoritmos

- Atendiendo a **la forma** en la que calculan la solución
 - Algoritmos **deterministas**: Siempre la misma solución para los mismos datos de entrada (instancia).
 - Algoritmos **probabilistas**: Pueden generar soluciones diferentes para los mismos datos de entrada (instancia).
- Atendiendo al **tipo de solución** :
 - Algoritmos **exactos**: Proporcionan la solución óptima.
 - Algoritmos **aproximados**: Proporcionan buenas soluciones con un grado de aproximación al optimo
 - Algoritmos **heurísticos**: Proporcionan buenas soluciones pero no podemos determinar si es la óptima.

Desarrollo e implementación de algoritmos

- **Modelar**

- ✓ Determinar los supuestos del problema y los datos de inicio.
- ✓ Determinar la estructura de datos que represente la información del problema
- ✓ No siempre es posible ajustarse al 100% de la realidad. Será necesario evaluar si con el modelo elegido podemos llegar a soluciones deseadas.

- **Diseñar**

- ✓ Aplicar técnicas conocidas para la resolución de problemas.

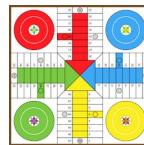
- **Analizar**

- ✓ Determinar las cantidades de operaciones que realizará el diseño realizado

Desarrollo e implementación de algoritmos. Modelar

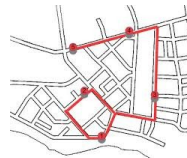
- **Modelos Matemáticos según los datos que se conocen**

- **Deterministas:** Todos los datos del problema se conocen
- **Estocásticos:** Hay datos del problema que nos se conocen



- **Modelos Matemáticos según cambia la realidad**

- **Estáticos:** El paso del tiempo no influye en el comportamiento de otras variables.



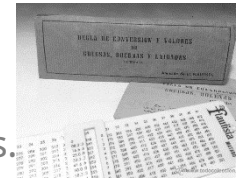
- **Dinámico:** El tiempo es una variable que influye en las variables.



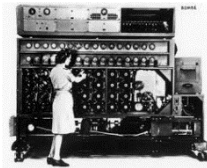
Desarrollo e implementación de algoritmos. Analizar(I)

- Determinar los recursos necesarios. ¿Cómo de bueno es nuestro algoritmo?
 - Eficiencia en **tiempo**.
 - Eficiencia en espacio
- Análisis **teórico** / Análisis práctico (independencia del procesador)
- Comparación. ¿Existen mejores algoritmos?
- Clasificación de los problemas:

Años 30 : Problemas computables y no computables.



Años 50 : Complejidad de los problemas computables(búsqueda de algoritmos más eficaces).

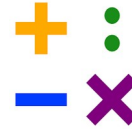


Años 70 : Clasificación de los problemas computables: P y NP.

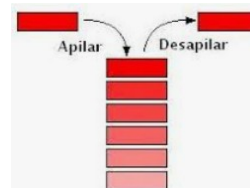
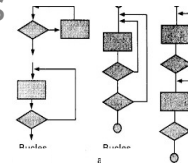


Análisis de la complejidad temporal (I)

- Mediremos la complejidad de un algoritmo por el **número de operaciones elementales(OE)** que realiza.
- ¿Cuáles son las operaciones elementales?
 - Suma, resta, multiplicación y división
 - Asignaciones
 - Condiciones , llamadas a funciones externas y retornos
 - Accesos a estructuras de datos



$a += b$	$a = a + b$
$a -= b$	$a = a - b$
$a *= b$	$a = a * b$
$a /= b$	$a = a / b$
$a \% = b$	$a = a \% b$



Análisis de la complejidad temporal (II)

- Orden de complejidad
 - ✓ Decimos que un algoritmo tiene orden de complejidad $O(f)$ si su tiempo de ejecución en operaciones elementales está acotado, salvo por una constante, por f para todos los tamaños de entrada a partir de un cierto n_0

Análisis de la complejidad temporal (III)

1

```
def codigo_1( number ):  
    a = 0  
    for j in range(1, number+1):  
        a += a + j  
  
    for k in range(number, 0, -1):  
        a -= 1  
        a *= 2  
    return a
```

Operaciones

Complejidad

2

```
def codigo_2():  
    a = 0  
    a -= 1  
    a *= 2
```

Operaciones

Complejidad

Análisis de la complejidad temporal (IV)

3

```
def codigo_3( number ):  
    a = 0;  
  
    for j in range(1, number+1):  
        for k in range(1, number+1):  
            a += a + ( k*j )  
  
    return a
```

Operaciones

Complejidad

4

```
def polinomico(number):  
    x = 0  
    for i in xrange(1,number):  
        for j in xrange(1,number):  
            x += i + j  
  
    for i in xrange(1,number):  
        for j in xrange(1,number):  
            for k in xrange(1,number):  
                x += i * j * k  
  
    return x
```

Operaciones

Complejidad

Análisis de la complejidad temporal (V)

Operaciones

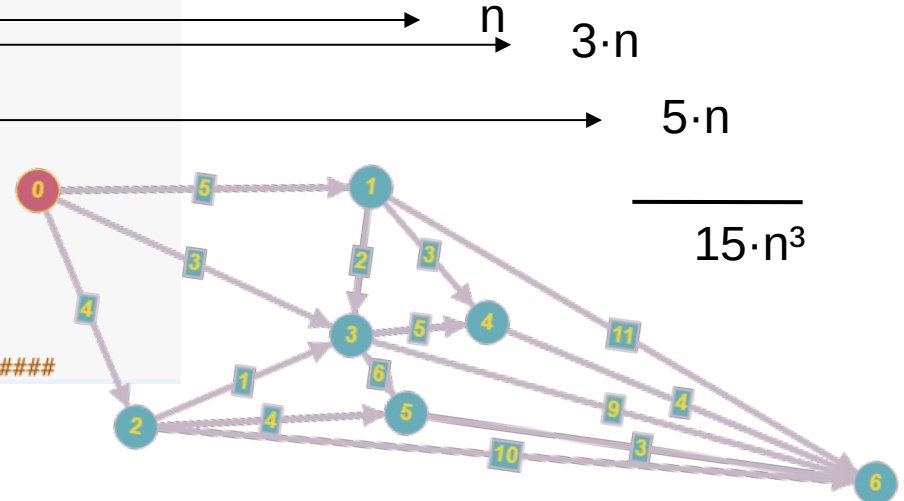
```
#####
def Precios(TARIFAS):
#####
    #Total de Nodos
    N = len(TARIFAS[0])

    #Iniciación de la tabla de precios
    PRECIOS = [ [9999]*N for i in [9999]*N]
    RUTA = [ [""]*N for i in [""]*N]

    for i in range(N-1):
        for j in range(i+1, N):
            MIN = TARIFAS[i][j]
            RUTA[i][j] = i

            for k in range(i, j):
                if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
                    MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
                    RUTA[i][j] = k
                    PRECIOS[i][j] = MIN

    return PRECIOS, RUTA
#####
```



$$C(i,j) = \min \{T(i,j) , C(i,k)+T(k,j) \text{ para todo } i < k \leq j \}$$

Análisis de la complejidad temporal (VI)

• Ecuaciones en recurrencia. Ejemplo 2(*)

```
#quick_sort
A = [9187, 244, 4054, 9222, 8373, 4993, 5265, 5470, 4519, 7182, 2035, 3506, 4337, 7580, 2554, 2824, 8357, 4447, 7379]

def quick_sort(A):
    if len(A) == 1:
        return A
    if len(A) == 2:
        return [min(A), max(A)]

    IZQ=[]
    DER=[]

    #pivote = (A[0]+ A[1] + A[2])/3
    pivote = sum(A)/len(A)

    #Falla en algunos casos, cuando hay valores repetidos y coinciden al principio de alguna lista
    #Buena opción. La mejor opción sería la mediana pero esto pasa por ordenar la lista
    # que es precisamente lo que queremos hacer.

    for i in A:
        if i <= pivote :
            IZQ.append(i)
        else:
            DER.append(i)

    return quick_sort(IZQ) + quick_sort(DER)

@calcular_tiempo
def QS(A):
    return quick_sort(A)

print(QS(A))
```

$$c=2$$

$$k=1$$

$$T(n) = 2 \cdot T(n/2) + n$$

$$a=2$$

$$b=2$$

Ecuaciones

$$T_n = \begin{cases} c \cdot n^k, 1 \leq n < b \\ a \cdot T\left(\frac{n}{b}\right) + c \cdot n^k, n \geq b \end{cases}$$

Soluciones

$$T(n) = O(n^k) \text{ si } a < b^k$$

$$T(n) = O(n^k \cdot \log(n)) \text{ si } a = b^k$$

$$T(n) = O(n^{\log_b(a)}) \text{ si } a > b^k$$

(*) El termino n-simo depende de un término(s) divisor

Análisis de la complejidad temporal (VII)

- Ordenes de complejidad (de menor a mayor)

$O(1)$	Orden constante	😄
$O(\log n)$	Orden logarítmico	😄
$O(n)$	Orden lineal	😄
$O(n \log n)$	Orden cuasi-lineal	😊
$O(n^2)$	Orden cuadrático	😊
$O(n^3)$	Orden cúbico	😊
$O(n^a)$	Orden polinómico	😊
$O(2^n)$	Orden exponencial	😱
$O(n!)$	Orden factorial	😱

Tabla 1

Comparación de los órdenes de complejidad según el tamaño del problema

n	$\log n$	n^2	2^n	$n!$
2	1	4	4	2
8	3	64	256	40.320
32	5	1.024	$4,3 \times 10^9$	$2,6 \times 10^{35}$
100	6	104	$1,2 \times 10^{27}$	$9,3 \times 10^{177}$

Algoritmos de Ordenación

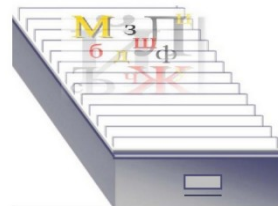
03MIAR – Algoritmos de optimización

Agenda

1. Evaluación y Entregas de la asignatura
2. Introducción a los algoritmos
3. Algoritmos de Ordenación
4. Técnicas de diseño de algoritmos
5. Problemas Tipo
6. Algoritmos de Búsqueda
7. Descenso del Gradiente
8. Algoritmos Heurísticos
9. Algoritmos Genéticos
10. Bibliografía

Algoritmos de Ordenación

- **Definición:** Ordenar una lista de valores numéricos
- Algoritmos:
 - ✓ Por Inserción
 - ✓ Por Selección
 - ✓ Burbuja
 - ✓ Mezcla
 - ✓ Montículos
 - ✓ Ordenación rápida (Quick Sort)
- Utilizaremos algunas **operaciones no elementales**:
 - ✓ Obtener el máximo o el mínimo de una lista : $O(n)$
 - ✓ Intercambiar dos valores de una lista: $O(1)$



Algoritmos de Ordenación(VII)

- Resumen de complejidades:

Algoritmo	Mejor	Peor	Medio	Espacio
Ordenar mezcla	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
Ordenación por inserción	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Ordenación por burbuja	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Ordenación rápida	$O(n \log(n))$	$O(n^2)$	$O(n \log(n))$	$O(1)^a$
Ordenación por montículos	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(1)$

^a En el mejor caso incluso es $\log(n)$.

Análisis comparativo de algoritmos de ordenamiento

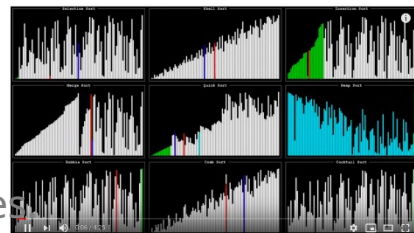
<https://pereiratechtalks.com/analisis-de-algoritmos-de-ordenamiento/>

- Visualización:

<https://www.youtube.com/watch?v=ZZuD6iUe3Pc>

- Otras ordenaciones

- ✓ Shell: mejora de inserción por comparación con varias posiciones.
- ✓ Burbuja bidireccional (cocktail sort). Ordena por los dos extremos



Técnicas de diseño de algoritmos

03MIAR – Algoritmos de optimización

Agenda

1. Evaluación y Entregas de la asignatura
2. Introducción a los algoritmos
3. Algoritmos de Ordenación
- 4. Técnicas de diseño de algoritmos**
5. Problemas Tipo
6. Algoritmos de Búsqueda
7. Descenso del Gradiente
8. Algoritmos Heurísticos
9. Algoritmos Genéticos
10. Bibliografía

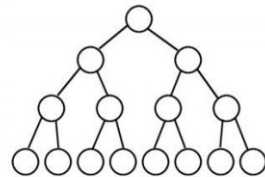
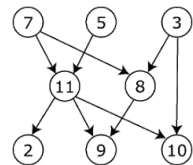
Técnica Voraz. Algoritmos voraces(I)

- **Definición:** Basada en la división del problema por etapas, eligiendo en cada etapa una decisión para construir la solución que resulte la más adecuada o ambiciosa sin considerar las consecuencias. Las decisiones descartadas serán descartadas para siempre. **Resumen: elegir en cada etapa la decisión optima.**
 - Resulta muy eficiente y aconsejable en muchos problemas donde por **características del problema**, toma en cada etapa la mejor decisión conduce a la optima.
 - Pero no en todos los casos se cumple! Deberemos demostrar formalmente que las mejores decisiones parciales llevan a la mejor solución final.
 - Otra dificultad. Encontrar la **función de selección** que determine la mejor decisión parcial.
 - Si se cumplen todas las condiciones favorablemente es la técnica mas recomendable por su eficiencia.
 - Es apropiada para problemas de optimización pero también la podemos usar para encontrar soluciones factibles cuando el espacio de soluciones es grande.

Grafos

Uso de grafos para modelizar problemas

- **Definición de grafo:** Es una pareja de conjuntos V (**vértices**) y A (**Aristas**) donde cada arista se compone de un par de vértices.
- Si el par de vértice (u,v) se considera diferente de (v,u) decimos que el **grafo es dirigido** y se representan las aristas con flechas. En caso contrario decimos que es no dirigido y las aristas se representan con líneas.
- Decimos que es **conexo** si todos los vértices están conectados directamente o por nodos intermedios.
- Decimos que es **ponderado** si todas las aristas tienen asociado un valor numérico.
- Definimos un **árbol** como un subconjunto de aristas que conectan todos los nodos de forma única.

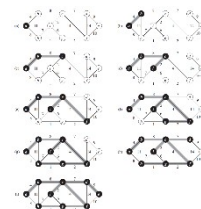


Grafos. Prim y Kruskal

- Existen dos algoritmos que proporcionan una solución para obtener el **árbol de recubrimiento mínimo(*)**.

Algoritmo Prim

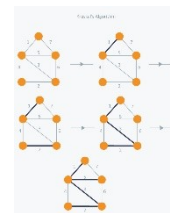
- Para cada vértice elegimos la arista de menor coste que une el subconjunto de vértices no seleccionados con el subconjunto de vértices seleccionados.
- El vértice del subconjunto de los no seleccionados pasa al conjunto de los seleccionados y se continua con uno nuevo.



Algoritmo Kruskal

- Se ordenan las aristas por orden decreciente.
- Se van seleccionando aristas de tal manera que no se formen ciclos (las aristas que formen ciclos se eliminan para no ser considerados en las siguientes etapas)

<https://i.stack.imgur.com/TTwpR.png>



Grafos. Prim y Kruskal

- Complejidad

Algoritmo Prim

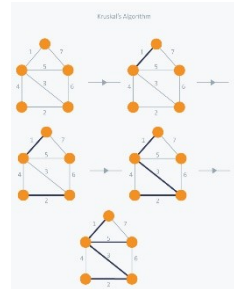
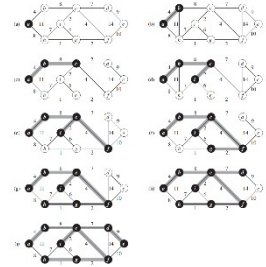
- Siempre $O(n^2)$.

Algoritmo Kruskal

- $O(a \cdot \log(n))$. Donde $a = n^0$ de aristas y $n = n^0$ de vértices

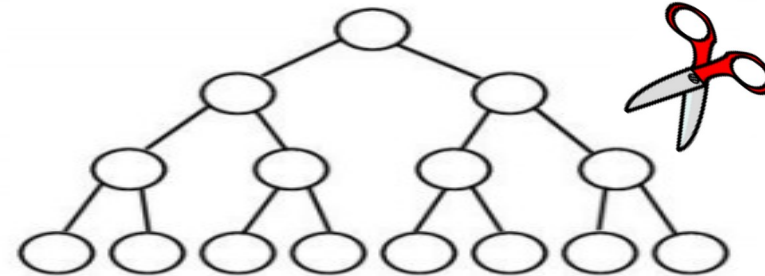
La elección de uno u otro dependerá del número total de aristas:

- ✓ Si hay muchas aristas (se aproxima a $n \cdot (n-1)$) la complejidad de Kruskal se acerca a $O(n^2 \cdot \log(n))$ peor que Prim
- ✓ Si hay pocas aristas (se aproximan a n) la complejidad de Kruskal se acercará a $O(n \cdot \log(n))$ mejorando a Prim



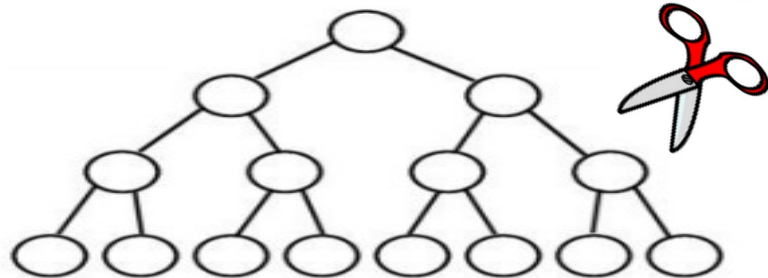
Vuelta Atrás. Backtracking (I)

- **Definición:** Construcción sistemática y por etapas de todas las soluciones posibles que pueden construirse mediante una tupla $(x_1, x_2, \dots, x_n)$ en la que cada componente x_i puede explorarse en la etapa i -ésima. A través de un **árbol de expansión** se modela todo el espacio de soluciones donde cada nodo es un valor diferente para cada elemento x_i .
- Características que permiten identificar problemas aplicables:
 - ✓ Problemas discretos en los que las soluciones se componen de elementos que pueden ser relacionado para expresarlos en un árbol de expansión.
 - ✓ Es posible encontrar un criterio para descartar determinadas ramas (ramificación y poda) para evitar la búsqueda bruta)



Vuelta Atrás. Backtracking (II)

- La eficiencia en general no será buena (exponencial $O(2^n)$) a menos que podamos incorporar mejoras para evitar la exploración en todos los nodos.
- La función de descarte debe ser tan sencilla en términos de complejidad como el coste de la exploración exhaustiva. En caso contrario no mejoraremos.
- El modelo de tener en cuenta las restricciones explícitas del problema. Nuestro diseño será mejor en la medida que seamos capaces de identificar e implementar las restricciones implícitas para



Divide y vencerás (I)

- **Definición:** Es posible dividir el problema principal recursivamente en sub-problemas más pequeños similares al original o sencillos de resolver.
- Características que permiten identificar problemas aplicables:
 - ✓ El problema puede ser descompuesto en problemas mas pequeños pero de la misma naturaleza que el principal.
 - ✓ Es posible resolver estos sub-problemas de manera recursiva o de otra manera sencilla.
 - ✓ Es posible combinar las soluciones de los sub-problemas para componer la solución al problema principal.
 - ✓ Los sub-problemas son disjuntos entre si. No hay solapamiento entre los sub-problemas.
 - ✓ Debemos asegurar que el proceso de divisiones recursivas finaliza en algún momento.

Divide y vencerás(VIII). Recursividad

- La recursividad permite diseños sencillos y legibles pero debemos tener en cuenta evaluar la complejidad en cuanto a uso de memoria o espacio en disco pues **en cada iteración multiplicamos el uso de recursos**.
- Solución a algunas ecuaciones en recurrencia:

Ecuaciones

$$T_n = \begin{cases} c \cdot n^k, 1 \leq n < b \\ a \cdot T\left(\frac{n}{b}\right) + c \cdot n^k, \wedge n \geq b \end{cases}$$

Soluciones

$$T(n) = O(n^k) \text{ si } a < b^k$$

$$T(n) = O(n^k \cdot \log(n)) \text{ si } a = b^k$$

$$T(n) = O(n^{\log_b(a)}) \text{ si } a > b^k$$

Programación dinámica (I)

- **Definición:** Es posible dividir el problema en subproblemas más pequeños, guardando las soluciones para ser utilizadas más adelante.
- Características que permiten identificar problemas aplicables:
 - ✓ Es posible almacenar soluciones de los subproblemas para ser utilizados más adelante
 - ✓ Debe verificar el **principio de optimalidad** de Bellman: *“en una secuencia optima de decisiones, toda sub-secuencia también es óptima”* (*)
 - ✓ La necesidad de guardar la información acerca de las soluciones parciales unido a la recursividad provoca la necesidad de preocuparnos por la complejidad espacial (cuantos recursos de espacio usaremos)

Problemas Tipo

03MIAR – Algoritmos de optimización

Agenda

1. Evaluación y Entregas de la asignatura
2. Introducción a los algoritmos
3. Algoritmos de Ordenación
4. Técnicas de diseño de algoritmos
- 5. Problemas Tipo**
6. Algoritmos de Búsqueda
7. Descenso del Gradiente
8. Algoritmos Heurísticos
9. Algoritmos Genéticos
10. Bibliografía

Programación lineal (I)

Definición: Son problemas de **programación lineal** aquellos para los que se necesita optimizar(maximizar o minimizar) una función objetivo que es lineal(*) (y por tanto continua) y multivariable sobre unas restricciones que vienen expresadas en igualdades o desigualdades también lineales.

- Existe muchos problemas que pueden ser enunciados en términos lineales(flujo, inventario, gestión de cartera, asignación de recursos, planificación...)
- **Resolución:** Método del **Simplex** y Método del Punto Interior.
- Dependiendo de la región que delimitan las restricciones el problema puede

Programación lineal entera (I)

- Son problemas de **programación lineal** en los que se exige que algunas o todas las variables sean **enteras**.
- Tipo
 - Puros: **todas** las variables deben ser enteras
 - Mixtos: **algunas** variables deben ser enteras
- **No existe un algoritmo** específico que los resuelva. Cada problema requiere un diseño específico según la técnica más adecuada.

Problema del agente viajero (TSP)

- El **TSP** (travelling salesman problem) se enuncia como la búsqueda optima del camino que debe realizar un viajero para recorrer un conjunto de ciudades de tal manera que la distancia recorrida sea mínima pasando una vez y solo una vez por cada una de las ciudades y comenzando y finalizando en la misma ciudad.
- El TSP es uno de los problemas con más estudios dentro de optimización matemática debido a que es muy habitual en el reparto de mercancías.
- El problema se modela con un grafo, donde los nodos son las ciudades y las aristas son las rutas que llevan asociado un valor que representa el coste o la distancia que hay que minimizar.

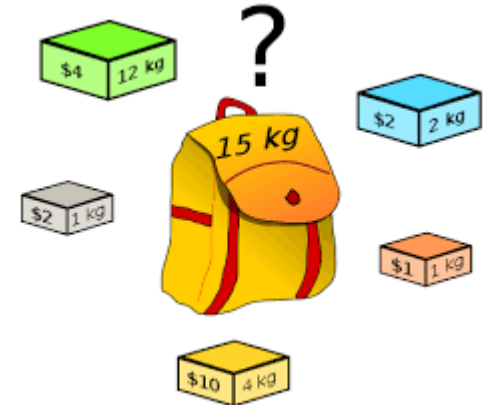
Problema de la mochila

- El **KP** (knapsack problem) se enuncia como la manera óptima de seleccionar un subconjunto de objetos que tienen asociado un valor y un peso cada uno de tal manera que maximicen el valor pero cuyo peso no supere un cierto límite.
- El KP también es uno de los problemas clásicos debido a que es modelo de muchas situaciones en la práctica.
- También es un problema difícil de resolver para datos de entrada grandes.

$$\text{Maximizar: } \sum_{i=1}^n v_i x_i$$

$$\text{Sujeto a: } \sum_{i=1}^n w_i x_i \leq W,$$

$$x_i = \begin{cases} 1 & \text{si se selecciona el objeto } i \\ 0 & \text{en caso contrario} \end{cases}, \quad i = 1, \dots, n$$



Otros problemas

- **Enrutamiento de vehículos (VRP).** TSP con varios agentes
- **Problema de la dieta** (Stigler, 1930): minimizar el gasto cubriendo necesidades nutricionales.
- Problema de **satisfacibilidad booleana (SAT)**. P.ej
Existe x_1, x_2, x_3, x_4 binarias (0,1) que den como resultado verdadero a:

$$(x_1 \vee \neg x_3) \wedge (\neg x_2 \vee x_3 \vee \neg x_4)$$

- **Problema de asignación:** asignar tareas a agentes para maximizar producción
- Compendio de problemas NP:

<http://web.archive.org/web/20200218105451/http://www.nada.kth.se/~viggo/wwwcompendium/>

Algoritmos de Búsqueda

03MIAR – Algoritmos de optimización

Agenda

1. Evaluación y Entregas de la asignatura
2. Introducción a los algoritmos
3. Algoritmos de Ordenación
4. Técnicas de diseño de algoritmos
5. Problemas Tipo
- 6. Algoritmos de Búsqueda**
7. Descenso del Gradiente
8. Algoritmos Heurísticos
9. Algoritmos Genéticos
10. Bibliografía

Algoritmos de Búsqueda (I)

- **Definición:** Técnicas que persigue la localización de uno o varios elementos con ciertas propiedades dentro de la estructura de datos.
- Dado que una cantidad importante de problemas se pueden modelar como estructuras de datos tipo árbol, es importante conocer técnicas para explorar arboles.
- Técnicas principales:
 - **Búsqueda en amplitud o anchura** (BFS): A* (+)
 - **Búsqueda en profundidad** (DFS): Ascenso de colina(hill climbing)(+)
 - **Ramificación y Poda**

Algoritmos de Búsqueda. (V)

• Resumen

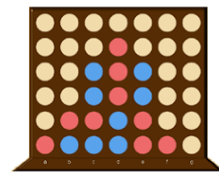
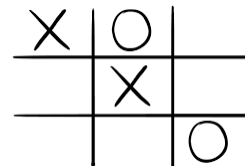
- En general los algoritmos de búsqueda son adecuados cuando el espacio de soluciones es acotado. Dejan de tener efectividad para arboles muy extensos.



- Ej. Tratar de resolver el juego del ajedrez por estos métodos resulta un poco ingenuo dado la **cantidad de posibilidades para los movimientos**.



- Sin embargo si es asequible resolverlos para juegos como 3 en raya o 4 en raya.



Descenso del gradiente

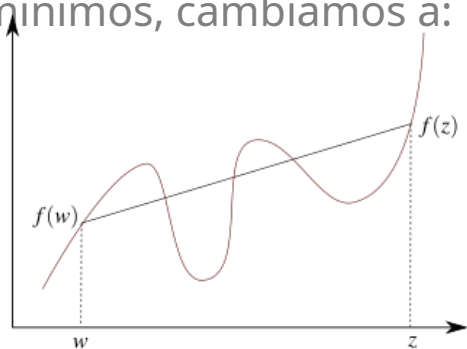
03MIAR – Algoritmos de optimización

Agenda

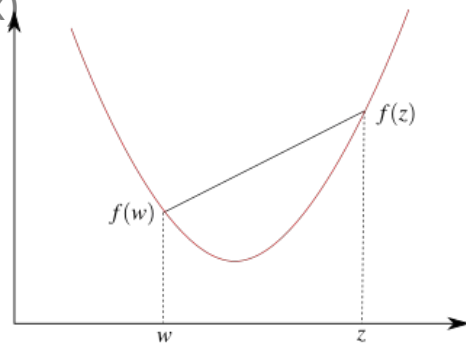
1. Evaluación y Entregas de la asignatura
2. Introducción a los algoritmos
3. Algoritmos de Ordenación
4. Técnicas de diseño de algoritmos
5. Problemas Tipo
6. Algoritmos de Búsqueda
7. **Descenso del Gradiente**
8. Algoritmos Heurísticos
9. Algoritmos Genéticos
10. Bibliografía

Descenso del Gradiente – AGD

- Es un algoritmo **iterativo** de optimización para encontrar valores mínimos para funciones multi-variables **convexas** y **diferenciables**.
- Convexidad
 - ✓ Permitir asegurar que la solución que encontremos es global frente a que solo sea local
 - ✓ ¿El segmento que une dos puntos está siempre por encima de la función?
 - ✓ Si es cóncava y buscamos mínimos, cambiamos a: $-f(x)$



a) Función no convexa



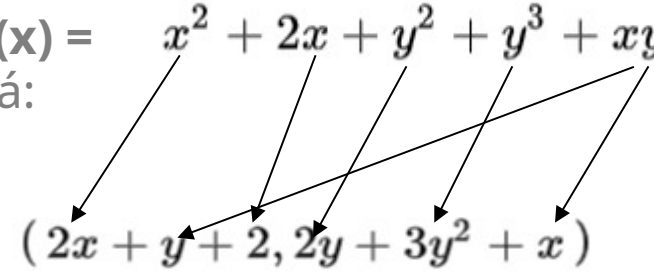
b) Función convexa

Descenso del Gradiente – AGD

- **Diferenciable:** derivable en n dimensiones
- **Gradiente:** Es un vector (diferente para cada punto = campo vectorial) cuyas coordenadas son las derivadas parciales:

$$\nabla f(\mathbf{r}) = \left(\frac{\partial f(\mathbf{r})}{\partial x_1}, \dots, \frac{\partial f(\mathbf{r})}{\partial x_n} \right)$$

Ejemplo: Dada la función $f(\mathbf{x}) = x^2 + 2x + y^2 + y^3 + xy$
el gradiente será:

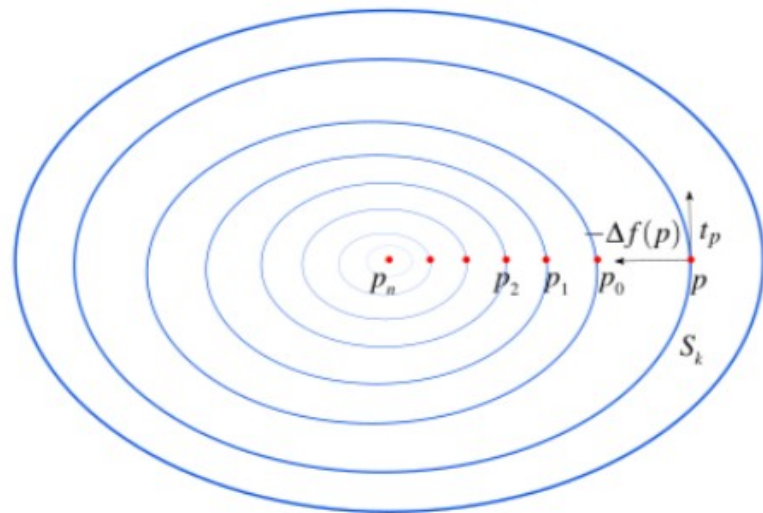
$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right) \quad (2x + y + 2, 2y + 3y^2 + x) \quad \text{En el punto } (x, y)$$


Descenso del Gradiente – AGD

- El procedimiento parte de un punto **p** como **solución aproximada** y da pasos en el sentido opuesto al gradiente (si minimizamos) de la función en dicho punto.

$$p_{t+1} = p_t - \alpha_t \nabla f(p_t)$$

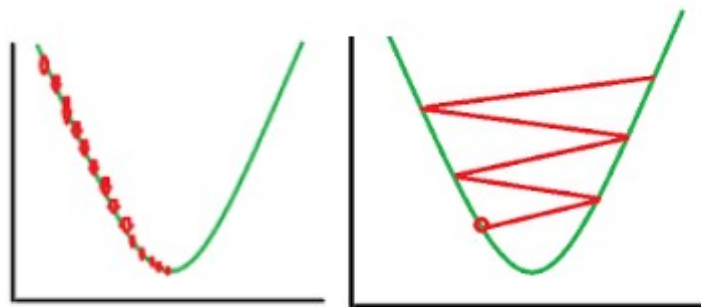
Donde el parámetro α_t se selecciona para que p_{t+1} sea solución
 y $f(p_t) \geq f(p_{t+1})$



Descenso del Gradiente – AGD

- La elección del parámetro α_t : llamado **tasa de aprendizaje (learning rate)** es importante para hacer efectivo el proceso de acercamiento a la solución óptima.
- Un valor demasiado alto puede provocar que el proceso no se acerque lo suficiente
- Un valor demasiado pequeño puede provocar exceso de iteraciones

$$p_{t+1} = p_t - \alpha_t \nabla f(p_t)$$



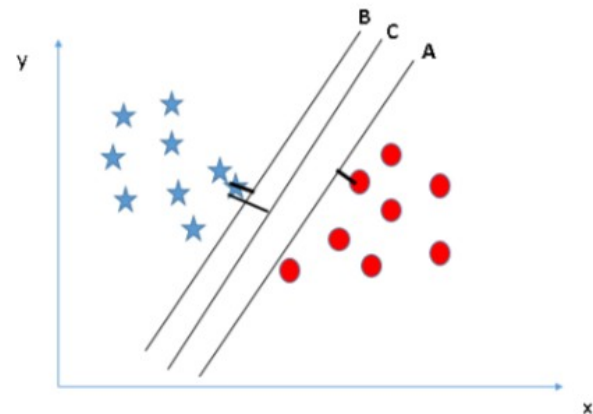
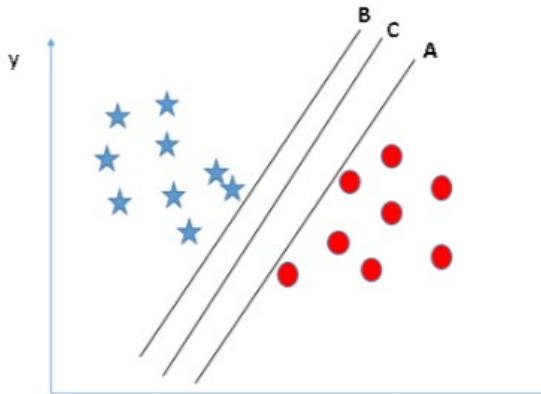
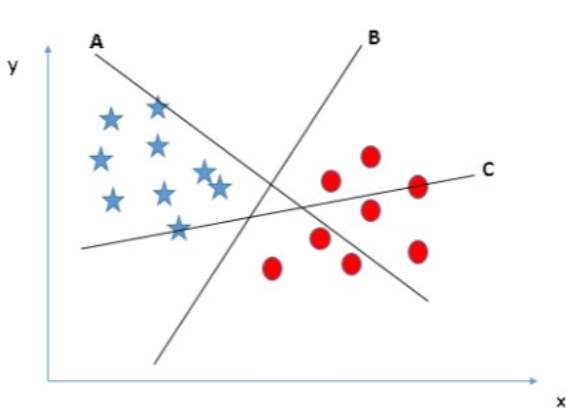
learning rate demasiado pequeño

learning rate demasiado grande

Descenso del Gradiente – GD

- Aprendizaje supervisado y la regresión lineal

¿Cómo identificar la mejor recta (hiperplano en N-dimensión) que separa los conjuntos de datos?

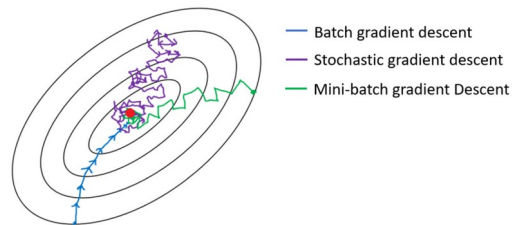


*Seleccionar el que esté
A mayor distancia*

Descenso del Gradiente – GD

- **Dependiendo del Volumen de Datos**

- ✓ Descenso del gradiente por lotes (**batch gradient descent**)
Calcula la desviación para todos los puntos!!!
- ✓ Descenso del gradiente estocástico(**stochastic gradient descent**)
Recalcula para cada punto.
- ✓ Descenso del gradiente por lotes reducido(**mini-batch gradient descent**)
Mezcla de ambos conceptos



Algoritmos Heurísticos

03MIAR – Algoritmos de optimización

Agenda

1. Evaluación y Entregas de la asignatura
2. Introducción a los algoritmos
3. Algoritmos de Ordenación
4. Técnicas de diseño de algoritmos
5. Problemas Tipo
6. Algoritmos de Búsqueda
7. Descenso del Gradiente
- 8. Algoritmos Heurísticos**
9. Algoritmos Genéticos
10. Bibliografía

Introducción a las metaheurísticas

- **Algoritmos aproximados**

Es posible encontrar soluciones próximas al óptimo en problemas complejos(*) en un tiempo razonable.

- **Destinados a usarlos cuando:**

- No hay un método exacto o requiere mucho tiempo(o memoria)
- No es imprescindible la solución optima pero se quiere una buena solución

- ¿Como puedo encontrar soluciones de alta calidad para una función de objetivo dada?

- Terminología

- Metaheurísticas: Se refiere a técnicas generales (directriz, estrategia,...)
- Heurístico: Se refiere a un algoritmo concreto.

(*)Problemas no abordables por métodos exactos (búsqueda exhaustiva, programación dinámica,...)

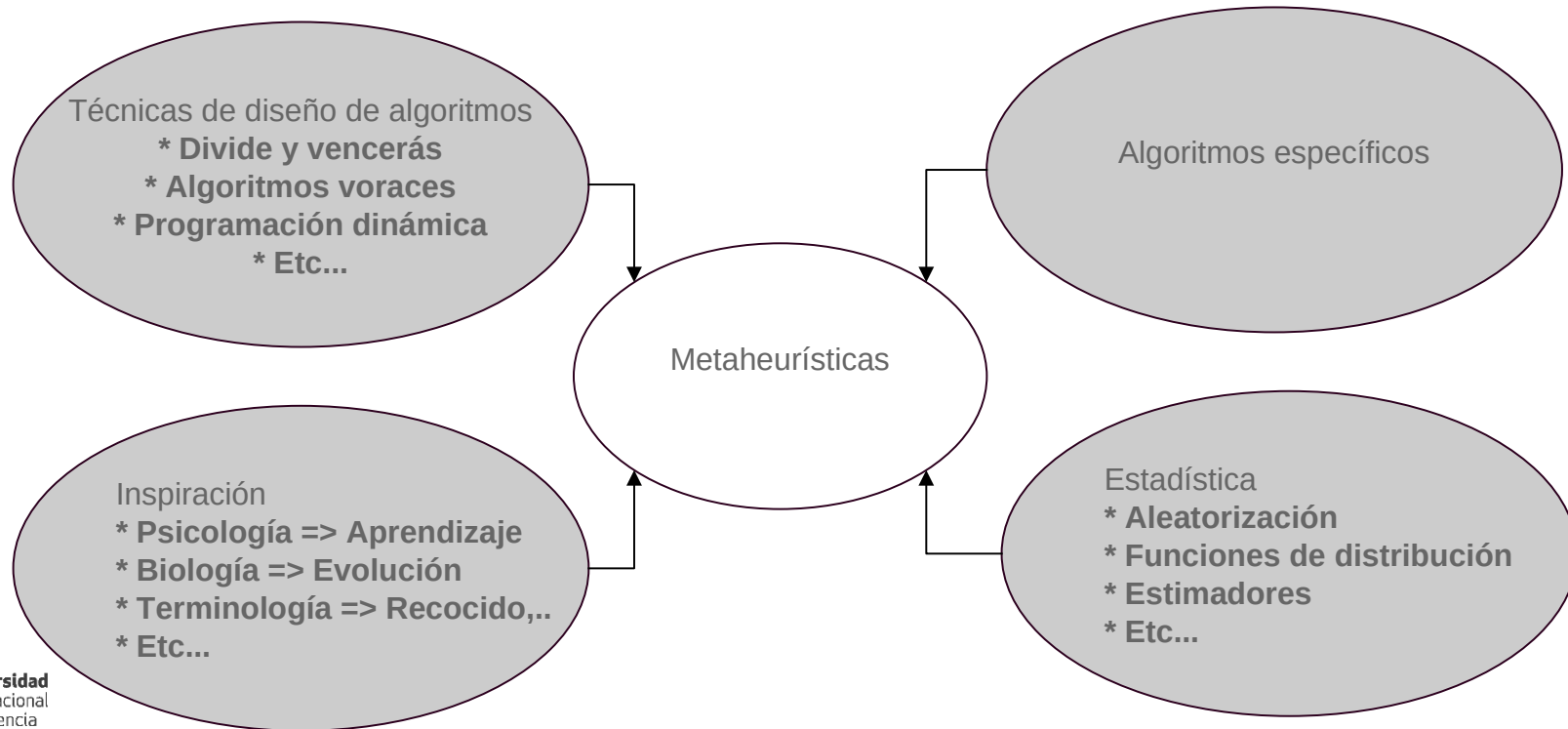
Introducción a las metaheurísticas

Definición. Metaheurísticas

- Técnicas de algoritmos aproximados, en general iterativos, que exploran el espacio de soluciones con una estrategia “inteligente” o de “sentido común” adaptada al problema.
- **Ventajas:**
 - Técnicas de propósito general
 - En general proporcionan buenos resultados
 - Son “fáciles” de implementar
 - Algunos admiten tratamiento en paralelo
- **Inconvenientes**
 - Son aproximados, no exactos
 - En general no son deterministas(probabilistas)
 - No existe base teórica(matemática) que los valide.

Introducción a las metaheurísticas

Elementos las de Metaheurísticas



Introducción a las metaheurísticas

Características de las Metaheurísticas

- **Intensificación:** Esfuerzo centrado en la búsqueda de la región actual
- **Diversificación:** Esfuerzo centrado en explorar otras regiones

Objetivo: Equilibrio entre Intensificación y Diversificación

- Identificar rápidamente regiones con soluciones de buena calidad
- No emplear mucho esfuerzo en regiones ya exploradas o poco prometedoras

Metaheurísticas de búsquedas

Búsqueda aleatoria

Definición: Es un proceso por el que se van generando soluciones aleatorias en cada iteración y se devuelve la mejor.

Inicio

GENERA(Solución Inicial)
Solución Actual $\leftarrow$ Solución Inicial;
Mejor Solución $\leftarrow$ Solución Actual;

Repetir

GENERA(Solución Actual);
Si Objetivo(Solución Actual) **es mejor que** Objetivo(Mejor Solución)
entonces Mejor Solución $\leftarrow$ Solución Actual;

Hasta (Criterio de parada);

DEVOLVER (Mejor Solución);

Fin

Generación aleatoria



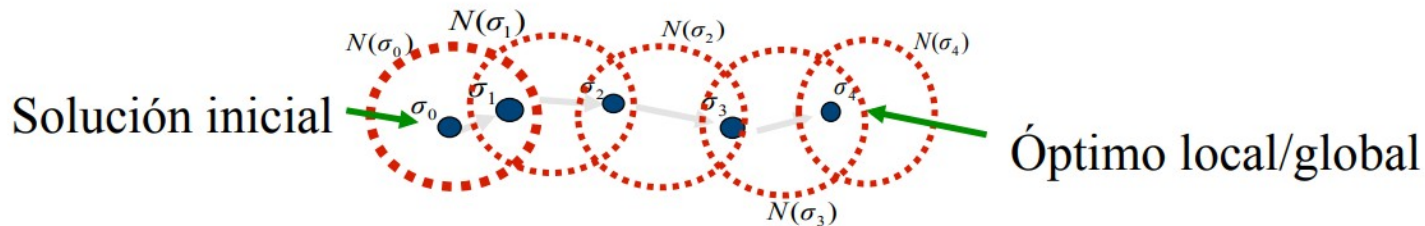
Metaheurísticas de búsquedas basadas en trayectorias

Búsqueda local

Definición: Es un proceso por el que dada una solución inicial, se va seleccionando iterativamente una nueva solución en el entorno local componiendo una trayectoria.

Elementos básicos:

- Establecer **solución inicial**
- Establecer una **codificación** para las soluciones
- Establecer un **operador de generación de vecino**(estructura de entorno)
- Establecer un **criterio de parada** para finalizar la iteración



Metaheurísticas: Búsqueda Tabú

Definición: Es un proceso de búsqueda por entornos con uso de **memoria adaptativa**.

- El termino proviene de comportamientos socioculturales en los que se establecen prohibiciones que desaparecen con el tiempo.
- La memoria adaptativa consiste en una lista tabú que permite:
 - **restringir** el entorno de búsqueda
 - **intensificar** sobre zonas de espacio de búsqueda ya visitadas
 - **diversificar** sobre zonas del espacio de búsqueda poco o nada visitadas

Metaheurísticas: Recocido simulado - SA

Definición: Es un proceso de búsqueda por entornos basado en la **aceptación probabilista** de soluciones.

- El termino proviene del tratamiento de metales y procesos termodinámicos. Se realizar un enfriamiento controlado con el objetivo de proporcionar determinada estructura al metal (simulated annealing).
- Aceptación probabilista se refiere a un control por el que la frecuencia de movimientos de escape(aquellos que dan lugar a peores soluciones) se va disminuyendo según una función de probabilidad.
- Diversificar al principio e intensificar al final.

Metaheurísticas. Métodos constructivos. Multiarranque

Definición: Es un proceso de búsqueda en el que se **repiten** los dos procesos principales de búsqueda, generación de **solución inicial** y **búsqueda local** hasta satisfacer un **criterio de parada** establecidos.

- **La soluciones iniciales** puede ser puramente aleatoria o “dirigidas” según las características del problema para que sean de buena calidad.
- Parar la búsqueda local en cada repetición se puede usar cualquier método de búsqueda.
- El criterio de parada puede establecerse en fijar el número de iteraciones en cada repetición o basado en la evolución de las soluciones.

Metaheurísticas. Métodos constructivos. GRASP

Definición: Es un proceso **multiarranque** de búsqueda **voraz**, **aleatorio** y **adaptativo** (*greedy randomized adaptative search procedures – GRASP, Feo y Resende, 1995*)

Se trata de un proceso combinado:

- **Voraz** se refiere a la construcción de soluciones iniciales por algún método voraz para que sean de calidad.
- **Aleatorio** se refiere a controlar la “voracidad”. En lugar de comportarse 100% voraz (probabilidad = 1 de elegir el mejor elemento para la solución) se añade una probabilidad sobre un conjunto de elementos candidatos.
- **Adaptativo** se refiere a que los beneficios del nuevo elemento se evalúan para condicionar la elección de los siguientes.

Metaheurísticas. Métodos constructivos. Colonias de hormigas

Definición: Es un proceso **multiagente** de búsqueda en el que cada agente se encarga de **construir probabilísticamente** soluciones aprovechando **información de otros agentes**.

(*Ant Colony Optimization – ACO, Marco Dorigo, 1992*)

Donde:

- **Multiagente** se refiere la utilización de varios agentes que realizan una determinada tarea de búsqueda basada en instrucciones sencillas.
- **Construcción probabilista** se refiere a que cada agente tomará decisiones según las instrucciones con alguna función de probabilidad.
- La información generada por cada agente es compartida para el resto de la comunidad de agente (feromona).



Algoritmos Genéticos

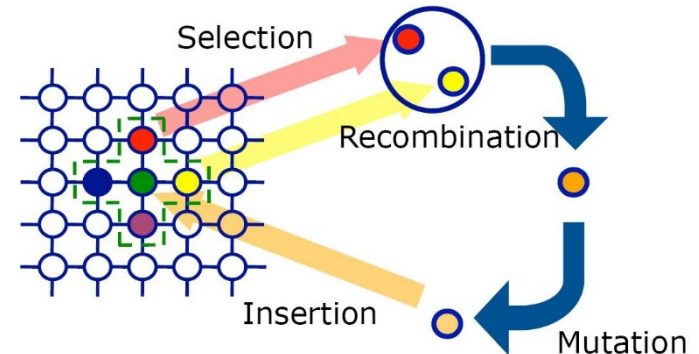
03MIAR – Algoritmos de optimización

Agenda

1. Evaluación y Entregas de la asignatura
2. Introducción a los algoritmos
3. Algoritmos de Ordenación
4. Técnicas de diseño de algoritmos
5. Problemas Tipo
6. Algoritmos de Búsqueda
7. Descenso del Gradiente
8. Algoritmos Heurísticos
- 9. Algoritmos Genéticos**
10. Bibliografía

Fundamentos de los algoritmos genéticos y evolutivos

- Se hace **evolucionar** una población de individuos (cada uno de los cuales puede representar una posible solución)
- La población se somete a **acciones aleatorias** semejantes a las de la evolución biológica (mutaciones y recombinaciones genéticas).
- Los individuos se seleccionan de acuerdo con una función de adaptación(**fitness**) en función de la cual se decide qué individuos sobreviven (los más adaptados) y cuáles son descartados (los menos aptos).



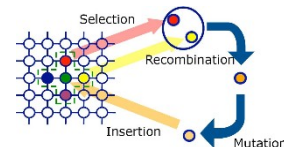
Fundamentos de los algoritmos genéticos y evolutivos

Esquema General

```
t ← 0
población(t) ← poblaciónInicial
EVALUAR(población(t))

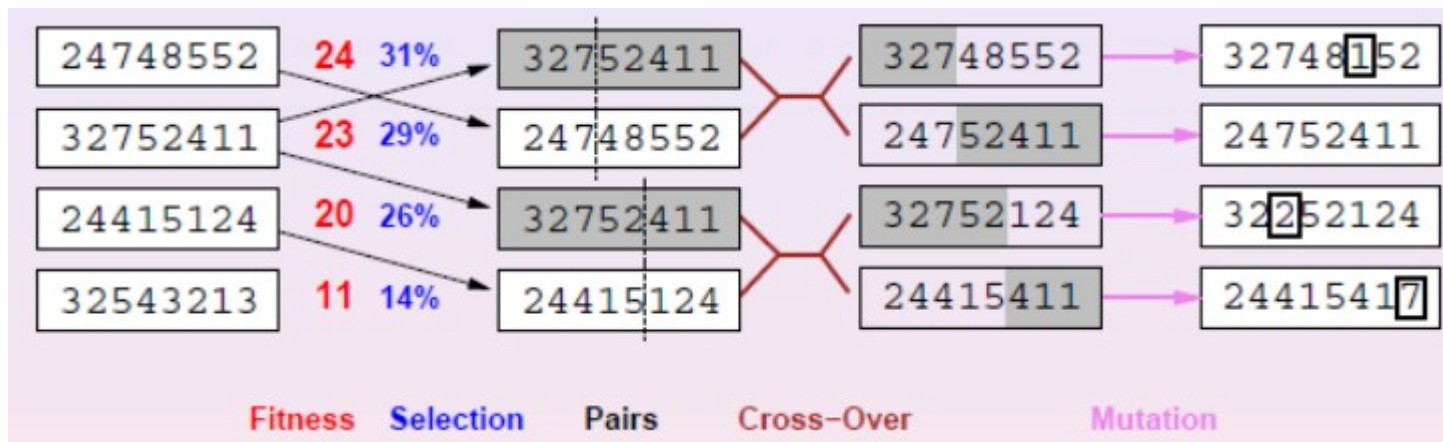
while not (condición de terminación)
    t ← t + 1
    población(t) ← SELECCIONAR(población(t-1))
    población(t) ← CRUZAR(población(t))
    población(t) ← MUTAR(población(t))
    EVALUAR(población(t))

return población(t)
```



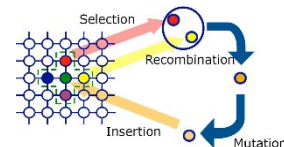
Fundamentos de los algoritmos genéticos y evolutivos

Esquema General. Selección, Cruce y Mutación



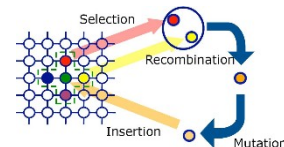
Características de los algoritmos genéticos y evolutivos

- Los algoritmos genéticos y evolutivos se utilizan para solucionar **problemas complejos**, pero **no garantizan la optimalidad**(técnicas heurísticas)
- En muchas ocasiones, ni siquiera podemos determinar cómo de cerca.
- Usan poblaciones de soluciones potenciales(en vez de un solo individuo), lo que hace menos probable de quedar **atrapadas en óptimos locales**.
- No necesitan disponer de **conocimiento específico** sobre el problema que intentan resolver (aunque puede ayudar).
- Usan **operadores probabilistas**, mientras las técnicas tradicionales(algoritmos exactos) utilizan operadores determinísticos.
- Se adaptan bien a problemas dinámicos(circunstancias cambiantes en el tiempo)



Componentes de un algoritmo genético/evolutivo

- **Representación** de las soluciones del problema.
- Función de evaluación (**fitness**/aptitud).
- Mecanismo de creación de la **población inicial**.
- **Operadores genéticos** (cruce & mutación).
- Mecanismo de **selección** [probabilística] (con elitismo).



Bibliografía y Recursos

03MIAR – Algoritmos de optimización

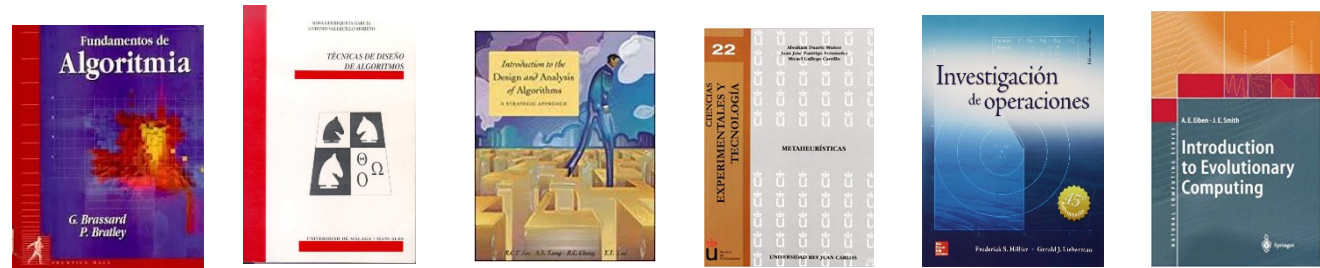
Agenda

1. Evaluación y Entregas de la asignatura
2. Introducción a los algoritmos
3. Algoritmos de Ordenación
4. Técnicas de diseño de algoritmos
5. Problemas Tipo
6. Algoritmos de Búsqueda
7. Descenso del Gradiente
8. Algoritmos Heurísticos
9. Algoritmos Genéticos
10. Bibliografía

Ampliación de conocimientos y habilidades

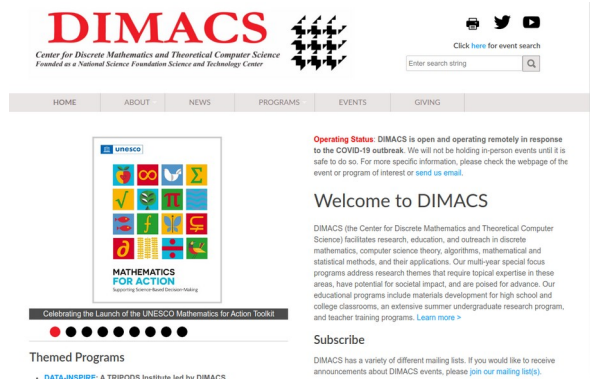
Bibliografía

- Brassard, G., y Bratley, P. (1997). Fundamentos de algoritmia
- Guerequeta, R., y Vallecillo, A. (2000). Técnicas de diseño de algoritmos.
<http://www.lcc.uma.es/~av/Libro/indice.html>
- Lee, R. C. T., Tseng, S. S., Chang, R. C., y Tsai, Y. T. (2005). Introducción al diseño y análisis de algoritmos.
- Duarte, A. (2008). Metaheurísticas. Madrid: Dykinson.
- Hillier, F. S., y Lieberman, G. J. (2015). Investigación de Operaciones.(Capítulo 14. Metaheurísticas)
- A.E. Eiben & J.E.Smith(2007). Introduction to Evolutionary Computing



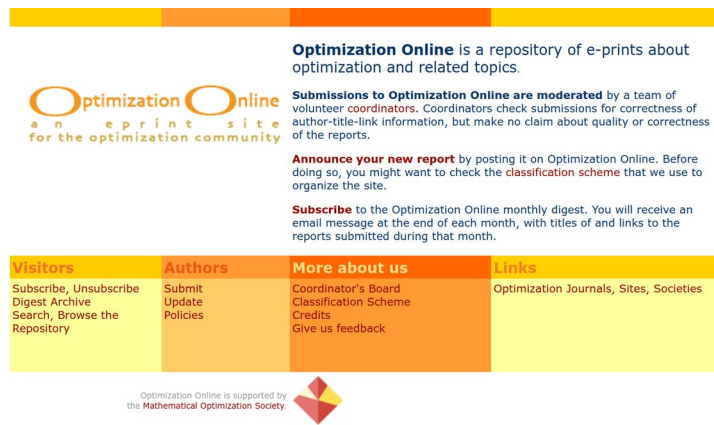
Ampliación de conocimientos y habilidades

- Centro de Matemáticas Discretas y Ciencias de la Computación Teórica, U. Rutgers
- Investigación y la divulgación en algoritmos, teoría de la computación, ...
- Programas de investigación("Special Focus"), talleres académicos, programas educativos,...



Ampliación de conocimientos y habilidades

- **Repositorio** online de acceso abierto dedicado al campo de la optimización
- Compartir trabajos antes de ser publicados formalmente en revistas científicas.



Optimization Online is a repository of e-prints about optimization and related topics.

Submissions to Optimization Online are moderated by a team of volunteer coordinators. Coordinators check submissions for correctness of author-title-link information, but make no claim about quality or correctness of the reports.

Announce your new report by posting it on Optimization Online. Before doing so, you might want to check the **classification scheme** that we use to organize the site.

Subscribe to the Optimization Online monthly digest. You will receive an email message at the end of each month, with titles of and links to the reports submitted during that month.

Visitors	Authors	More about us	Links
Subscribe, Unsubscribe Digest Archive Search, Browse the Repository	Submit Update Policies	Coordinator's Board Classification Scheme Credits Give us feedback	Optimization Journals, Sites, Societies

Optimization Online is supported by the **Mathematical Optimization Society**.

Ampliación de conocimientos y habilidades(II)

Material de la asignatura

- ☐  Materiales del docente
 - ☐  Presentaciones de Clase
 - ☐  Bibliografía(Libros en la Biblioteca Virtual)
 - ☐  Otros Materiales
 - ☐  Videos de Expertos
 - ☐  Complejidad Algorítmica
 - ☐  Combinatoria
 - ☐  Python
 - ☐  Programación Lineal(LP)
 - ☐  Descenso del Gradiente
 - ☐  Traveling Salesman Problem(TSP)
 - ☐  Recursos Web
 - ☐  Artículos

Actividad. Encuesta de satisfacción

Actividades



Encuesta satisfacción alumno con la asignatura y el profesor

La Universidad Internacional de Valencia tiene como objetivo conocer y analizar el grado de satisfacción y expectativas del alumnado con respecto a la gestión de las titulaciones que ofrece con la finalidad de incrementar el grado de satisfacción en cada edición y contribuir con vuestra información a la mejora continua.

A través de esta encuesta recogeremos vuestra opinión sobre el desarrollo de cada una de las asignaturas y el desempeño de los docentes que han participado.

Vuestra valoración es un elemento fundamental del sistema de calidad y mejora permanente de la Universidad, por lo que solicitamos vuestra colaboración.

¿Preguntas?



Gracias

raul.reyero@professor.universidadviu.com